

Design & Implementation of AES-128 Encryption in Verilog HDL

Abstract: Hardware implementation of AES-128 in Verilog for FPGA deployment. Verified RTL, testbench-driven functional validation, resource & timing analysis on Xilinx Vivado 2023.2.

❏ Significance: Demonstrates secure, low-latency block cipher suitable for embedded and FPGA systems.

Introduction

- Why security matters

Confidentiality, integrity, and low-latency encryption for embedded systems, IoT, and secure comms.

- Problem Statement

Implement AES-128 in synthesizable Verilog, optimize for FPGA, and verify correctness and timing.

- What is AES?

Symmetric block cipher standardized by NIST — widely used for secure data exchange.

- Objectives & Scope

- Design modular RTL for AES-128
- Key expansion and round logic
- Functional verification & resource analysis on FPGA

AES-128 at a glance



Key Size

128-bit symmetric key



Block Size

128-bit data block (16 bytes)



Rounds

10 rounds (encryption)



Benefits

Strong security, hardware-friendly S-box, well-documented primitives

Encryption Process & Key Expansion



Round ops: SubBytes (nonlinear S-box), ShiftRows (byte permutation), MixColumns (Galois field mix), AddRoundKey (XOR with round key). KeyExpansion derives 11 round keys.

System Design & Architecture

Architecture Overview

Top-level AES-128 system for streaming 128-bit blocks with control FSM and key input interface

1

Top Module

Interfaces: clk, rst, start,
data_in[127:0], key_in[127:0],
data_out[127:0], done

2

Control FSM

Manages round sequencing,
start/ready handshake, and
pipeline control

3

Data Path

Round module instances, SBox
lookups, MixColumns GF(2⁸)
ops

4

KeyExpansion

Generates 11 round keys,
supports on-the-fly or
precomputed modes

Module Breakdown



AES_128Core

Top-level integrator — instantiates round logic and FSM.



Round / FinalRound

Executes SubBytes, ShiftRows, (MixColumns), AddRoundKey.



SBox

Combinational ROM-based or algebraic implementation for SubBytes.



KeyExpansion

Rcon, RotWord, SubWord — outputs round_key[0..10].



Support

AddRoundKey, MixColumns (GF multiply by {02},{03}), ShiftRows mapping

Module hierarchy: Top → AES_128Core → {KeyExpansion, Round x9, FinalRound, SBox, AddRoundKey}

Implementation Details

- Language: Verilog (synthesizable subset)
- FPGA toolchain: Xilinx Vivado 2023.2
- Implementation styles: Iterative (single-round) and pipelined options
- Clock domain: single clock, synchronous reset
- Key storage: register file or BRAM for precomputed keys



Testbench

Stimulus generator, known-answer tests (NIST vectors), random vectors, error injection

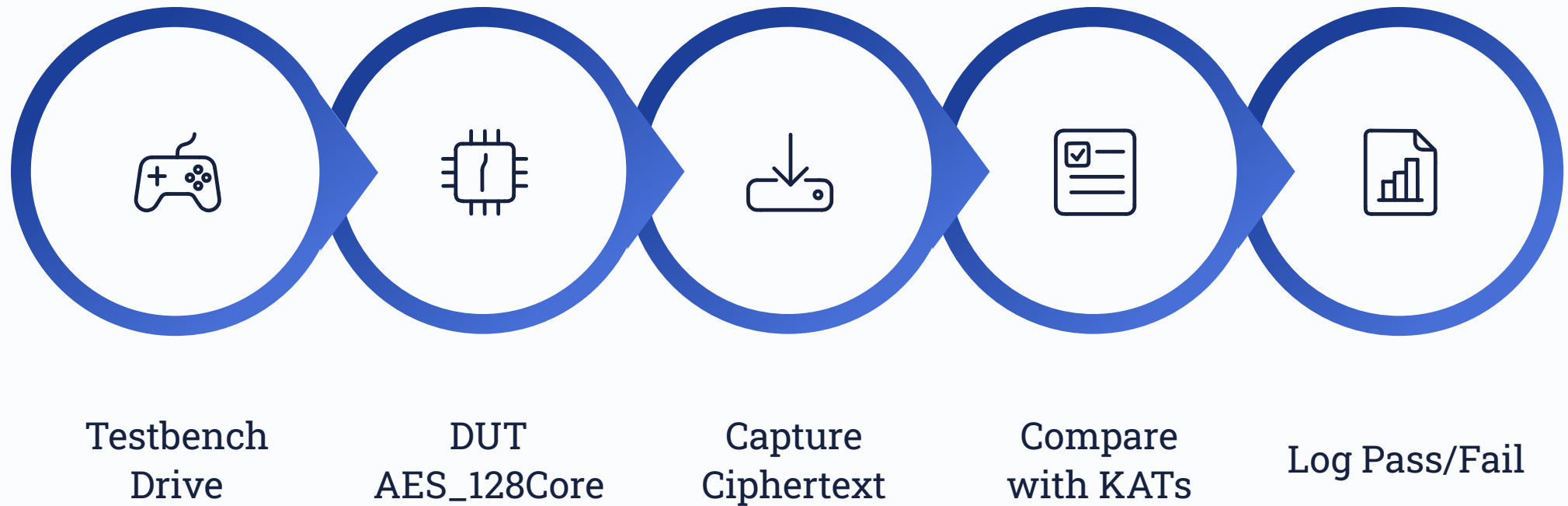


Simulation

Functional simulation (ModelSim/Simulator), waveform capture, assertion checks



Simulation & Verification



Verification checklist:

- Passed NIST AES test vectors (ECB mode)
- Waveform captures for each round (internal state observation)
- Assertions for ready/done handshake and key validity
- Corner-case tests: all-zeros key/plaintext, single-bit flips

Results – RTL, Resources & Timing

Key deliverables:

- RTL schematic (visualized in Vivado)
- Resource utilization: LUTs, FFs, BRAM, DSPs
- Timing summary: Fmax, path slack, setup/hold margins
- Performance: latency (cycles/block) and throughput (Mbps at target Fmax)

 Placeholders: include RTL screenshots, utilization table, and timing report in final report appendix.

Note: Populate exact numbers after synthesis/place & route on target device.

Applications, Conclusion & Next Steps



Applications

Secure comms, embedded devices, secure boot, VPN appliances, IoT endpoints



Advantages

Deterministic latency, hardware acceleration, compact area on FPGA



Limitations

Side-channel leakage risk, tradeoffs: area vs throughput vs pipeline depth



Future Work

Masking/side-channel countermeasures, AES-GCM integration, multi-block pipelining, formal verification

References: NIST FIPS-197, Xilinx Vivado docs, Verilog HDL reference. Appendix: Verilog source, testbench, waveforms, RTL images, timing reports.

Post-Optimization Results — LUT, FF, Timing & Power

Resource and timing comparison after merging the duplicate **SubBytes** block that previously lived inside the always-running **LastRound** instance.

Metric	Before merge	After merge	Δ (saving)
Flip-Flops (FF)	—	—	≈ 128 FFs removed
LUTs	—	—	91 LUTs removed
WNS (slack)	+6.061 ns	+6.381 ns	+0.320 ns (better)
Critical path	3.939 ns	3.619 ns	−0.320 ns
Estimated Fmax	~254 MHz	~276 MHz	+22 MHz
Total power	0.18 W	0.18 W	unchanged

Why the FF savings match exactly

Each **SubBytes** block contains 16 SBox instances, and every SBox has an 8-bit registered output. That gives one SubBytes = $16 \times 8 = 128$ flip-flops. Removing the duplicate SubBytes that used to sit inside the always-running LastRound instance saves almost exactly those 128 FFs. The accompanying **91 LUT** reduction comes from the combinational logic that surrounded it inside LastRound. The merge worked exactly as predicted.

Timing — WNS actually improved

WNS went from **+6.061 ns** (before merge) to **+6.381 ns** (after merge) — it got slightly *better*, not worse. The new mux added by the merge is a tiny piece of combinational logic that did not lengthen the critical path, and removing the parallel LastRound instance reduced routing pressure enough for the placer to find a better arrangement. The critical path is now $10 - 6.381 = 3.619$ ns, giving an estimated **Fmax ≈ 276 MHz** — a bit better than the ~254 MHz estimate before the merge.

Power — unchanged, and that is expected

Total power stayed at **0.18 W** before and after. The 130 FFs and 91 LUTs removed are a small fraction of the total, and **62%** of the power is device static leakage (**0.112 W**) which is fixed regardless of what the design does.

Note for the report: power confidence is still **Low** because Vivado is estimating switching activity generically rather than from the actual simulation. The number is real enough to report, but that caveat is worth one sentence in the write-up.